

Rapport formlang

Rapport de projet — formlang

Binôme : AGBESSI John Ulrich · LATOOUNDI Gregoire **Dépôt Git :**
<https://github.com/John271200/formLang-project> · **Commit final :** ff9b113

Tous les chiffres de ce rapport sont issus de **nos propres exécutions** (machine Windows 10, Python 3.11.9). Les bornes de complexité sont démontrées ou référencées (§ 4.4).

0. Résumé (½ page)

Le projet construit une unique bibliothèque `formlang/` qui gravite la hiérarchie de Chomsky : automates finis et transducteurs (régulier), automate à pile et grammaires (hors-contexte), automates d'arbres ascendants (langages d'arbres réguliers), machines de Turing et machine universelle (récursivement énumérable), puis referme la boucle théorique avec la congruence de Myhill–Nerode. Quatre applications **instancient** ce cœur sans jamais le réécrire : `apps/morpho` (classification morphologique par BUTA), `apps/shield` (pare-feu 100 % structurel : FST de normalisation → AFD de détection → PDA de délimiteurs → BUTA de décomposition), `apps/hashcons` (partage de structure en DAG sur `formlang.tree.Term`) et `apps/mtu` (calculatrice unaire dont + et - sont de vraies tables de machines de Turing, exécutées aussi *via* la machine universelle). `pipeline.py` orchestre le tout.

Résultat global de la suite de tests :

```
$ py -m pytest -q
.....
[100%]
37 passed in 0.35s
```

37 tests verts : les 28 tests de base (dont le bonus P4.5, `test_shield_double_encodage_P45`) plus **9 tests bonus** que nous avons écrits nous-mêmes (`tests/test_bonus.py`) pour valider chacun des bonus réalisés : construction de Thompson (J1), reconnaissance CYK (J2), infixation et minimisation d'un BUTA (J3), table monolithique de la multiplication (J4), mesure hash-consing sur ~10 000 mots (J5, § 5.3).

1. Étape régulier & transducteurs (Jour 1)

1.1 Q1.1 — Expression régulière de L_1

Avec $\Sigma = \{a, o, r\}$ et $L_1 = \{w \mid w \text{ contient le facteur } or\}$:

$$L_1 = (a|o|r)^* or (a|o|r)^* \quad (\text{soit } \Sigma^* \text{ or } \Sigma^*)$$

Justification. Tout mot dénoté contient l'occurrence explicite du facteur *or* (au milieu du motif) : il appartient à L_1 . Réciproquement, si $w \in L_1$, alors $w = x \cdot or \cdot y$ avec $x, y \in \Sigma^*$; x est engendré par le premier $(a|o|r)^*$, y par le second. La regex capture donc exactement L_1 .

1.2 Q1.2 — De l'AFN à l'AFD minimal

(a) AFN (on « devine » le début du facteur) :

```
q0 --a,o,r--> q0      (boucle)
q0 --o--> q1          (choix non déterministe sur 'o')
q1 --r--> q2
q2 --a,o,r--> q2      (absorbant, accepteur)
```

(b) Déterminisation (sous-ensembles). On part de $\{q_0\}$ et on ferme :

état AFD	sur o	sur a	sur r	remarque
$S_0 = \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$	initial
$S_1 = \{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_2\}$	o armé
$S_2 = \{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_2\}$	accepteur
$S_3 = \{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_2\}$	accepteur

(c) **Minimisation (Moore).** Partition initiale $\{S_0, S_1\} / \{S_2, S_3\}$ (non-finaux / finaux).
Raffinement 1 : dans $\{S_0, S_1\}$, $S_0 \xrightarrow{r} \text{non-final}$ mais $S_1 \xrightarrow{r} \text{final} \Rightarrow$ on sépare $\{S_0\} / \{S_1\}$. Dans $\{S_2, S_3\}$, les deux vont vers des finaux sur toutes les lettres $\Rightarrow$ bloc stable.
Raffinement 2 : plus rien ne bouge. Partition finale : $\{S_0\} \{S_1\} \{S_2, S_3\}$ — trois classes.

(d) AFD minimal (3 états, $A = S_0$, $B = S_1$, $C = S_2 \equiv S_3$) :

δ	a	o	r
A (init)	A	B	A
B	A	B	C
C (final)	C	C	C

C'est exactement la table de `_DFA_OR` dans `apps/shield/detector.py`. **Minimalité vérifiée par le code** : `detector_dfa().minimize().num_states()` renvoie **3** (test `test_minimisation_3_etats` vert), et le § 5.2 retrouve ces 3 états comme classes de Myhill–Nerode — c'est la preuve de minimalité.

1.3 Q1.3 — Facteur vs sous-séquence

$L_1' = \{w \mid w \text{ contient un } o \text{ suivi (pas forcément immédiatement) d'un } r\}$:

$$L_1' = \Sigma^* o \Sigma^* r \Sigma^*$$

Différence : dans L_1 le r doit suivre **immédiatement** le o (facteur) ; dans L_1' n'importe quels symboles peuvent s'intercaler (sous-séquence). **Inclusion** $L_1 \subseteq L_1'$: si $w = x \cdot o \cdot r \cdot y$, alors $w = x \cdot o \cdot \varepsilon \cdot r \cdot y$ est de la forme $\Sigma o \Sigma r \Sigma^*$ avec le bloc du milieu vide ; donc $w \in L_1'$.

L'inclusion est stricte : **oar** $\in L_1' \setminus L_1$ (un a s'intercale ; notre AFD répond bien `contains_or("oar") = False`, cf. `test_detector_or`).

1.4 E1.4 — Idempotence du FST leet (preuve)

Le transducteur `leet_fst` a un seul état q_0 (final) et les règles $4 \rightarrow a$, $3 \rightarrow e$, $0 \rightarrow o$, $1 \rightarrow i$, $5 \rightarrow s$, identité ailleurs. Soit T sa fonction. Pour toute lettre c de $T(w)$: soit c provient d'une règle leet (alors $c \in \{a, e, o, i, s\}$, qui n'est **pas** dans le domaine $\{4, 3, 0, 1, 5\}$ des règles), soit c a été copiée à l'identité (alors $c \notin \{4, 3, 0, 1, 5\}$ non plus). Donc chaque lettre de $T(w)$ est laissée invariante par T : **$T(T(w)) = T(w)$** . Vérifié : `T(4tt4ck) = attack`, `T(r0le) = role`, `T(T(4tt4ck)) = T(4tt4ck)` (`test test_leet_idempotent`).

1.5 Miroir : one-way impossible, two-way possible

Supposons qu'un transducteur séquentiel one-way à k états réalise $w \mapsto w^R$. La première lettre émise doit être la **dernière** lettre lue de w ; or après avoir lu le préfixe de longueur n , la machine ne dispose que de k états pour distinguer les $|\Sigma|^n$ préfixes possibles, tous à restituer intégralement plus tard. Dès que $|\Sigma|^n > k$ (pigeonhole), deux préfixes distincts $u \neq v$ mènent au même état ; les sorties sur u^R et v^R deviennent alors identiques à suffixe égal — contradiction. Une mémoire **bornée** ne peut pas retenir un mot **non borné**. Avec une tête two-way, il suffit d'aller à la fin du ruban et de le relire de droite à gauche (c'est `reverse_twoway`). Première limite due à la mémoire — elle annonce la pile (Jour 2) puis le ruban libre (Jour 4).

1.6 BONUS — Construction de Thompson (regex $\rightarrow$ AFN explicite)

`formlang/thompson.py` : analyseur par descente récursive (`alt := cat ('|' cat)*`, `cat := star*`, `star := base ('*' | '+' | '?')`, `base := '(' alt ')' | lettre`) ; chaque construction relie les fragments uniquement par des **ε -transitions** (union, concaténation, étoile — schémas de Thompson classiques, cf. Sipser 2012). Le résultat instancie `formlang.nfa.NFA` ($\varepsilon = ""$), donc bénéficie gratuitement de `_eps_closure` et `to_dfa`. **La boucle du jour 1 se referme entièrement en code :**

```
thompson("(a|o|r)*or(a|o|r)*").to_dfa().minimize().num_states() == 3
```

regex $\rightarrow$ AFN (Thompson) $\rightarrow$ AFD (sous-ensembles) $\rightarrow$ minimal (Moore) : on retrouve les **3 états**. Tests : `test_thompson_equivaut_au_detecteur` (équivalence avec l'AFD du détecteur sur 13 mots), `test_thompson_afn_afd_minimal_3_etats`, `test_thompson_operateurs`.

2. Hors-contexte (Jour 2)

2.1 Q2.1 — Non-régularité de $D = \{[n]^n \mid n \geq 0\}$ (pompage)

Supposons D régulier ; soit p sa constante de pompage. Choisissons $w = [p]^p \in D$, $|w| = 2p \geq p$. Considérons **toute** découpe $w = xyz$ avec $|xy| \leq p$ et $y \neq \varepsilon$. Comme les p premières lettres de w sont toutes des $[$, $x = [^a$ et $y = [^b$ avec $b \geq 1$ et $a+b \leq p$. Pompons avec $i = 2$:

$$xy^2z = [^{a(2b)} [^{(p-a-b)}]^p = [^{(p+b)}]^p, \quad b \geq 1$$

Ce mot a $p+b$ ouvrantes et p fermantes : $xy^2z \notin D$, ce qui contredit le lemme pour toutes les découpes admissibles. Donc **D n'est pas régulier.** ■

Conclusion pour le projet : le SingularityDetector (AFD, mémoire finie) ne peut pas vérifier l'imbrication — il faudrait « compter » un nombre non borné d'ouvrantes. D'où l'automate à pile du Jour 2.

2.2 Q2.2 — Grammaire des prompts bien parenthésés

Sur $\Sigma_3 = \{a, o, r, [,], (,)\}$:

$$S \rightarrow S S \mid [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$$

Exemple de dérivation de $[a(r)]$:

$$S \Rightarrow [S] \Rightarrow [SS] \Rightarrow [aS] \Rightarrow [a(S)] \Rightarrow [a(r)]$$

2.3 Q2.3 — Ambiguïté et désambiguïisation

G est ambiguë. Le mot aaa admet (au moins) deux arbres, selon l'associativité choisie pour $S \rightarrow SS$:

$$\text{arbre 1 : } S(S(S \rightarrow a, S \rightarrow a), S \rightarrow a) \quad ((aa)a)$$

$$\text{arbre 2 : } S(S \rightarrow a, S(S \rightarrow a, S \rightarrow a)) \quad (a(aa))$$

Grammaire non ambiguë équivalente (liste droite de blocs T) :

$$S \rightarrow T S \mid \varepsilon$$

$$T \rightarrow [S] \mid (S) \mid a \mid o \mid r$$

Chaque mot se décompose de façon **unique** en suite de blocs de tête T ; chaque T est déterminé par sa première lettre $\Rightarrow$ un seul arbre par mot.

2.4 E2.1 — PDA : fonction de transition

`DelimiterPDA.accepts` réalise le PDA à un état q , acceptation **pile vide** :

lu	sommet	action
[ou (	—	empiler
] (resp.))	[(resp. ()	dépiler

lu	sommet	action
] (resp.))	autre ou pile vide	rejet immédiat
a, o, r, e	—	ignorer

Fin de mot : accepter ssi la pile est vide. C'est la comparaison au **sommet** qui attrape l'imbrication croisée ([]) (le test `test_delimiters_pda` le vérifie), ce qu'un simple compteur ne ferait pas.

2.5 E2.2 — Génération bornée : le piège des formes non terminales

Notre `CFG.generate(max_len)` explore les formes sententielles (expansion du non-terminal le plus à gauche) avec **deux bornes** : nb de terminaux $\leq \text{max_len}$ et nb de non-terminaux $\leq 2 \cdot \text{max_len} + 2$. Sans la seconde, $S \rightarrow SS$ engendre $S, SS, SSS, \dots$ (0 terminal chacune) et la recherche diverge. La borne est sûre : une dérivation d'un mot de longueur $\leq \text{max_len}$ dans cette grammaire n'a jamais besoin de plus de non-terminaux simultanés (chaque S produit au plus 2 S et la longueur finale est bornée). Mesure réelle : `generate(4)` produit **257 mots**, contenant "", [], (), [a], [[]] et aucun mot déséquilibré (`test_cfg_engendre_des_mots_equilibres` vert).

2.6 BONUS — Reconnaissance par CYK

`CFG.cyk(w)` (dans `formlang/cfg.py`) : la grammaire est d'abord mise en **forme normale de Chomsky** (`_cnf` : START, élimination des ϵ -productions via l'ensemble des annulables, clôture des productions unitaires, TERM, BIN), puis l'appartenance est décidée par la table de programmation dynamique classique en $O(n^3 \cdot |G|)$ (Hopcroft-Motwani-Ullman 2006). Le mot vide est traité à part (axiome annulable). **Validation croisée** : CYK et le PDA du jour 2 donnent le **même verdict sur les 400 mots** de longueur ≤ 3 de Σ_3 (`test_cyk_equivalent_au_pda`) — deux modèles très différents (grammaire vs pile), une même classe de langages. Test complémentaire : `test_cyk_mots_equilibres` ([a(r)] ✓, ([]) X, "" ✓).

3. Arbres (Jour 3) — pivot

3.1 Définition utilisée et règles Δ

Un BUTA est $A = (Q, \Sigma, Q_f, \Delta)$ avec des règles $f(q_1, \dots, q_n) \rightarrow q$; l'exécution étiquette l'arbre en **post-ordre** (feuilles $\rightarrow$ racine) et accepte si la racine reçoit un état de Q_f . Notre moteur (`formlang/tree.py`) est **déterministe** : `delta` est un dictionnaire (symbole, états_enfants) $\rightarrow$ état, donc au plus une règle par clé ; l'absence de règle vaut REJECT, propagé vers le haut.

Δ morpho ($Q_f = \{\text{WORD}\}$) :

<code>prefix()</code> $\rightarrow$ PRE	<code>root()</code> $\rightarrow$ ROOT	<code>suffix()</code> $\rightarrow$ SUF	<code>nil()</code> $\rightarrow$ NIL
<code>prefixes</code> (PRE,NIL) $\rightarrow$ PREFS		<code>prefixes</code> (PRE,PREFS) $\rightarrow$ PREFS	
<code>suffixes</code> (SUF,NIL) $\rightarrow$ SUFS		<code>suffixes</code> (SUF,SUFS) $\rightarrow$ SUFS	
<code>rest</code> (ROOT,NIL) $\rightarrow$ REST		<code>rest</code> (ROOT,SUFS) $\rightarrow$ REST	
<code>word</code> (NIL,REST) $\rightarrow$ WORD		<code>word</code> (PREFS,REST) $\rightarrow$ WORD	

Δ **shield** ($Q_f = \{\text{DANGER}\}$, états $\text{SAFE} < \text{OVR} < \text{ROLE} < \text{DANGER}$) :

```
txt()→SAFE  enc()→SAFE  ovr()→OVR  role()→ROLE
seq(x,y) → DANGER si DANGER ∈ {x,y} ou {x,y} = {OVR,ROLE},
              sinon max de sévérité (16 règles, produit cartésien)
frame(x), sys(x) → SAFE si x = SAFE, DANGER sinon (8 règles)
```

Au total : 28 règles pour le shield, 13 pour la morpho.

3.2 Q3.1 — Déterminisme & coût

Dans les deux automates, chaque clé (symbole, tuple d'états enfants) apparaît **une seule fois** (c'est structurellement garanti par le dictionnaire Python : un `add_rule` ultérieur écraserait le précédent — et nos constructions n'ont aucun doublon). Conséquence : l'exécution est **unique** (pas de choix à explorer), et on visite chaque nœud exactement une fois avec un coût $O(1)$ par nœud (un accès de dictionnaire) : **coût total $O(n)$** pour un arbre à n nœuds.

3.3 Preuve d'unité (règle « gate »)

Les deux applicationsinstancient le **même** moteur — aucun automate réécrit :

```
# apps/morpho/automaton.py
from formlang.tree import Term, TreeAutomaton
def morpho_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"WORD"})
    A.add_rule("prefix", (), "PRE") ...

# apps/shield/decomposer.py
from formlang.tree import Term, TreeAutomaton, product
def shield_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"DANGER"})
    A.add_rule("txt", (), SAFE) ...
```

De même `apps/hashcons/store.py` importe `formlang.tree.Term` et reconstruit des `Term` au round-trip. Le pipeline (Jour 5) n'appelle que les apps.

3.4 Q3.2 — Pourquoi un AFD de mots échoue

Deux arbres de **même frontière** na pend et de classes différentes :

Arbre 1 (PREFIXED):	Arbre 2 (SUFFIXED):
<code>word(prefixes(prefix na, nil),</code>	<code>word(nil, rest(root na,</code>
<code>rest(root pend, nil))</code>	<code>suffixes(suffix pend, nil)))</code>

Un AFD de mots ne lit que la frontière (la suite des feuilles) : il reçoit la même entrée `napend` dans les deux cas et répond nécessairement la même chose. La classe morphologique dépend de la **structure**, invisible dans le `yield`. Le BUTA, lui, lit l'arbre entier : il étiquette `prefix na` → `PRE` dans l'arbre 1 mais `root na` → `ROOT` dans l'arbre 2.

3.5 Q3.3 — Théorème du yield

Un arbre accepté par `morpho_automaton` a nécessairement la forme `word(P, rest(root, S))` où `P` est une chaîne (éventuellement vide) de `prefixes` terminée par `nil` et `S` une chaîne de `suffixes` terminée par `nil` (ce sont les seules règles menant à `WORD`). Sa frontière est donc

`prefix* · root · suffix*` c'est-à-dire $p^* r s^*$

— précisément un **langage régulier de mots**, celui de l'étage 1. C'est le pont : un mot est un arbre unaire, un AFD est un BUTA « tout en arité 1 », et l'ensemble des frontières d'un langage d'arbres régulier est hors-contexte (ici, il retombe même dans le régulier car les chaînes sont unaires).

3.6 Q3.4 — Réduplication

La reduplication copie une partie de la racine : le langage des mots $\{ww \mid w \in \Sigma^*\}$ en est l'abstraction. Ce langage n'est **pas** hors-contexte (pompage CFL : sur $a^p b^p a^p b^p$, toute découpe $uvxyz$ avec $|vxy| \leq p$ pompe dans une fenêtre trop étroite pour maintenir l'égalité des deux moitiés). Or les frontières des langages d'arbres réguliers sont exactement les langages hors-contexte : un langage d'arbres régulier ne peut donc pas imposer la contrainte de copie sur sa frontière — la reduplication **sort** du cadre BUTA. C'est un phénomène réel : Culy (1985) montre que le vocabulaire du bambara (constructions du type *wulu-o-wulu*) n'est pas hors-contexte.

3.7 Traces d'exécution ascendante (états calculés par notre run)

Morpho — `build_word(["a", "na"], "pend", ["a"])` (accepté, CIRCUMFIXED) :

```
word                                → WORD ∈ Q_f ✓
├ prefixes                          → PREFS
│   ├── prefix a                    → PRE
│   └─ prefixes(prefix na → PRE, nil → NIL) → PREFS
└─ rest                             → REST
    ├── root pend                   → ROOT
    └─ suffixes(suffix a → SUF, nil → NIL) → SUFS
```

Morpho — **arbre mal formé** `word(nil, rest(suffix er, nil))` : `suffix → SUF` mais aucune règle `rest(SUF, NIL) ⇒ REJECT` à `rest`, propagé ⇒ `INVALID` (test `test_morpho_rejet`).

Shield — `sys(seq(txt, frame(role)))` (bloqué) :

```
sys                                → DANGER ∈ Q_f ✓ BLOQUÉ
└─ seq                            → DANGER (un enfant DANGER)
    ├── txt                        → SAFE
    └─ frame(role→ROLE) → DANGER (frame d'un contenu non SAFE)
```

Shield — `seq(ovr, role)` : `ovr→OVR, role→ROLE, seq(OVR, ROLE)→DANGER` (la paire critique) ⇒ bloqué. À l'inverse `role()` isolé → `ROLE ∉ Q_f` ⇒ OK.

3.8 E3.4/E3.6 — Produit (intersection) et P4.5

$\text{product}(A_1, A_2)$: états = paires, finals = $Q_{f1} \times Q_{f2}$, et pour chaque couple de règles de même symbole et même arité, la règle produit $f((q_1, q'_1), \dots) \rightarrow (r_1, r'_1)$. Un arbre est accepté ssi les **deux** exécutions acceptent : $L = L(A_1) \cap L(A_2)$ (clôture par intersection des langages d'arbres réguliers). Application P4.5 : `enc_automaton` compte les feuilles `enc` (plafonné à 2, final {2}, 19 règles) et

```
dangerous_and_double_encoded = product(shield_automaton(), enc_automaton())
```

donne un automate de **172 règles** (mesuré) qui bloque ssi l'arbre est dangereux **et** porte ≥ 2 encodages. Vérifié : `sys(seq(enc, seq(enc, role)))` accepté ; `sys(role)` (dangereux mais 0 enc) et `seq(enc, enc)` (2 enc mais sûr) rejetés (test `test_shield_double_encodage_P45`).

3.9 BONUS — Infixation (s<um>ulat)

L'infixation (tagalog *s<um>ulat* « écrire ») coupe la racine en deux autour de l'infixe. Extension **purement déclarative** de Δ (aucune modification du moteur ni des règles existantes) : nouveau constructeur `iroot(left, inf, right)` et deux règles ajoutées à l'automate de base :

```
infix() → INF          iroot(ROOT, INF, ROOT) → ROOT
```

Une racine infixée se comporte ensuite comme une ROOT ordinaire — `rest`, `word`, préfixes et suffixes fonctionnent inchangés. Le test `test_infixation` prouve (1) que l'arbre de *s<um>ulat* est **rejeté** par l'automate de base (l'extension était nécessaire), (2) qu'il est accepté et classé INFIXED par l'automate étendu, (3) que les mots déjà couverts gardent exactement leur classe (aucune régression : *a-na-pend-a* reste CIRCUMFIXED).

3.10 BONUS — Minimisation d'un BUTA (quotient par congruence)

`formlang.tree.minimize(A)` implémente la congruence de Myhill–Nerode **pour les arbres** (TATA 2007, ch. 1) : deux états sont équivalents s'ils donnent le même verdict dans **tout contexte** (arbre à trou). Algorithme : partition initiale finaux/non-finaux, puis raffinement — la signature d'un état q est l'ensemble des contextes à un trou (symbole, position, blocs des autres enfants, bloc du résultat) des règles où q apparaît ; une règle absente vaut REJECT, donc un contexte présent d'un seul côté sépare aussi. À point fixe, on quotiente (états = blocs, règles projetées). **Test** : sur l'automate unaire « contient or » où l'état C a été dupliqué en D (rôles croisés $C \leftrightarrow D$), `minimize` fusionne C et D : **4 états** → **3 états**, et le quotient reconnaît le même langage que l'original **et** que l'AFD du jour 1 sur 8 mots-arbres unaires (`test_minimisation_buta`) — la boucle « un mot est un arbre unaire » se referme, et le lien avec le hash-consing est direct : partager, c'est identifier des sous-structures équivalentes.

4. Calculabilité (Jour 4)

4.1 E4.1 — MT générique : invariant et trace

`TuringMachine.run` maintient l'invariant : (*ruban partiel tape:dict (les cases absentes valent le blanc), position head, état state*) est exactement la configuration de *M* après *steps* pas. Chaque itération lit `tape.get(head, blank)`, applique δ (écrire / bouger L,R,S / changer d'état) ; arrêt sur état accepteur, état de rejet ou transition absente ; une garde `max_steps` lève une exception en cas de boucle (l'arrêt n'est pas décidable — § 4.5 — donc on **borne**). Trace réelle de **ADD sur 11+1** ($2+1=3$) :

pas	état	lit	action
0	q0	1	avance à droite
1	q0	1	avance
2	q0	+	+ devient 1 , passe en q1
3	q1	1	avance
4	q1	_	blanc : demi-tour, q2
5	q2	1	efface le dernier 1 , qf
6	qf	_	accepté ; ruban 111 = 3 ✓

(6 pas, `accepted=True` — sortie exacte de `ADD.run("11+1", trace=True)`.)

4.2 Q4.1/E4.2 — Encodage $\langle M \rangle$ et machine universelle

`encode(M)` linéarise la table en JSON canonique : liste **triée** de quintuplets $[q, a, q', b, d]$ + `start`, `accept` (trié), `reject` (trié), `blank`, avec `sort_keys=True`. **Injectivité** : deux machines distinctes diffèrent par au moins un champ ; le JSON canonique (ordre fixé des clés et des quintuplets) diffère alors aussi. **Décodabilité** : `decode` reconstruit chaque champ ; `decode(encode(M))` a exactement les mêmes transitions, états et symboles — round-trip exact, vérifié par `test_round_trip_encodage`. Mesure : $|\langle \text{ADD} \rangle| = 223$ caractères.

`UniversalTM.run( $\langle M \rangle$ , w)` réalise le principe du **programme enregistré** : elle décode $\langle M \rangle$ (la table devient une donnée) puis simule *M* sur *w*. **Correction (Q4.2)** par récurrence sur le nombre de pas : l'invariant est que la configuration simulée après *k* cycles égale la configuration de *M* après *k* pas ; au pas *k*+1, *U* lit le même symbole, consulte la même entrée de table, écrit/bouge/change d'état pareillement. Vérification expérimentale : `U( $\langle \text{ADD} \rangle$ , "111+11")` donne ruban 11111 et le même `accepted` que l'exécution directe (`test_test_utm_simule_comme_la_machine`).

4.3 E4.3 — Tables ADD/SUB ligne par ligne

ADD (fusion $\leftrightarrow 1$ puis retrait d'un surplus) :

transition	rôle
$(q0,1) \rightarrow (q0,1,R)$	traverser le bloc m

transition	rôle
$(q0,+) \rightarrow (q1,1,R)$	le séparateur devient un 1 : les blocs fusionnent ($m+n+1$ uns)
$(q1,1) \rightarrow (q1,1,R)$	traverser le bloc n
$(q1,) \rightarrow (q2,,L)$	blanc final : demi-tour
$(q2,1) \rightarrow (qf,,S)$	effacer le 1 excédentaire $\rightarrow m+n$ uns, accepter

SUB (appariement : effacer un 1 de n à droite, un 1 de m à gauche) :

transition	phase
$(q0,1/-) \rightarrow (q0,.,R) ; (q0,) \rightarrow (q1,,L)$	aller à l'extrémité droite
$(q1,1) \rightarrow (q2,,L)$	effacer un 1 de n
$(q1,-) \rightarrow (qf,,S)$	n épuisé : effacer -, il reste $m-n$ uns ✓
$(q2,1/-) \rightarrow (q2,.,L) ; (q2,) \rightarrow (q3,,R)$	revenir à l'extrémité gauche
$(q3,1) \rightarrow (q0,,R)$	effacer le 1 apparié de m, reboucler
$(q3,-) \rightarrow (q5,,R)$	m épuisé avant n ($m < n$) : passer au nettoyage
$(q5,1) \rightarrow (q5,,R) ; (q5,) \rightarrow (qf,,S)$	tout effacer : résultat 0 (troncature)

Le point délicat annoncé par l'énoncé (« la fin de m se lit par un blanc, pas par un compteur ») est visible en $q3$: c'est le **ruban** qui dit si m est épuisé. Mesures : SUB("111-11") $\rightarrow$ ruban 1 en 30 pas ; SUB("11-11") $\rightarrow$ ruban vide en 25 pas.

MUL/DIV par composition (c'est ce que permet la thèse de Church–Turing) : multiplication(n,m) = m additions successives *via la MT ADD* ; division(n,m) = soustractions répétées *via la MT SUB* $\rightarrow$ (quotient, reste), avec ZeroDivisionError si $m = 0$. **Résultats des tests exhaustifs** : test_calculatrice_exhaustif vérifie les 49 couples $(n,m) \in [0,6]^2$ pour +, -, $\times$ et les 42 couples pour $\div$ — tous exacts ; chainer(2, [+1, $\times 2$, -1, $\div 2$]) = 2 ; le tout **vert**. L'interpréteur universel (E4.4) refait l'addition en passant par U : addition_via_utm(3,2) = 5, soustraction_via_utm(5,2) = 3 (test test_interpreteur_universel).

4.4 Q4.3 — Surcoût de simulation (sourcé)

- Simulation par U d'une machine M pendant t pas : chaque cycle consulte la table $\langle M \rangle$, d'où un surcoût $O(t \cdot |\langle M \rangle|)$ (Arora & Barak 2009, ch. 1, « efficient universal Turing machine »).
- Réduction multi-ruban $\rightarrow$ mono-ruban : ralentissement au plus **quadratique**, $O(t^2)$ (Hopcroft, Motwani & Ullman 2006 ; Arora & Barak 2009, claim 1.6).
- Hennie & Stearns (1966) : simulation de k rubans sur **2 rubans** en $O(t \log t)$ — le ralentissement logarithmique de référence.

Aucune de ces bornes n'est « de mémoire » : références ci-dessus, § 7 de l'énoncé (bibliographie). Notre U, implantée au méta-niveau (décodage JSON puis délégation au simulateur), a un surcoût réel d'un décodage **unique** au départ ; la boucle de simulation est ensuite la même que l'exécution directe — cohérent avec le modèle $O(t \cdot |\langle M \rangle|)$ où la consultation de table est ici $O(1)$ amorti grâce au dictionnaire.

4.5 Q4.4 — Indécidabilité : « universel $\neq$ omniscient »

Soit HALT-U le problème « U s'arrête sur l'entrée $\langle M \rangle \# w$ ». Réduction depuis le problème de l'arrêt : étant donné (M, w) , construire l'entrée $\langle M \rangle \# w$ — c'est calculable (c'est notre encode). Par définition de U, U s'arrête sur $\langle M \rangle \# w$ **ssi** M s'arrête sur w. Un décideur de HALT-U déciderait donc l'arrêt, indécidable (Turing 1936). Donc **HALT-U est indécidable**. La machine universelle peut simuler *toutes* les machines, mais ne peut pas *prédire* leur arrêt : c'est précisément pourquoi notre run porte un `max_steps`.

4.6 BONUS — Table monolithique de la multiplication

MUL (`apps/mtu/machines.py`) est une **seule table** de 33 transitions sur l'alphabet de travail $\Gamma = \{1, *, X, Y, =, \sqcup\}$ — aucune composition, aucune boucle Python. Entrée $1^n * 1^m$, sortie : un ruban ne contenant que $1^{n \times m}$. Algorithme en phases :

phase	états	rôle
init	q_i, q_r	écrire le séparateur = en fin de ruban, revenir au début
marquage	q_0	marquer X le prochain 1 du bloc n (ou passer au nettoyage sur *)
copie	$q_1\text{--}q_5$	pour chaque 1 du bloc m : marquer Y, aller écrire un 1 après =, revenir
démarquage	q_6, q_7	restaurer les Y en 1 (le bloc m ressortira au cycle suivant)
nettoyage	q_8, q_9	effacer les X, le bloc m et = ; seul le résultat subsiste

C'est l'invariant « le ruban compte, pas un compteur » poussé à bout : la fin du bloc n se lit par *, la fin d'un cycle de copie par =, la fin du nettoyage par = aussi. **Validée par le simulateur**, doublement : `test_mul_monolithique_directe` vérifie les **25 couples** $(n, m) \in [0, 4]^2$ (y compris $n=0$ et $m=0$) via `TuringMachine.run`, et `test_mul_monolithique_via_utm` exécute MUL **par la machine universelle** : $U(\langle \text{MUL} \rangle, "111*11")$ laisse bien 6 uns sur le ruban.

5. Intégration & Myhill–Nerode (Jour 5)

5.1 E5.1 — Un exemple bout-en-bout

```
$ py pipeline.py --word 4or
    normalisé(FST) : aor          ← leet_fst (formlang.fst), 4→a
    facteur_or(AFD) : True        ← _DFA_OR (formlang.dfa), A→a→A→o→B→r→C
∈ F
    délimiteurs_ok(PDA) : True    ← DelimiterPDA (formlang.pda), pile vide

$ py pipeline.py --morpho mufafak (corpus_A)
{'classe(BUTA)': 'PREFIXED'}      ← discover a appris PRE=
{mu,ba,ki,vi,li,ma,wa,tu},
                                segment_to_tree('mufafak') =
word(prefixes(mu),
                                rest(fafak)), classify → PREFIXED
```

La chaîne mot traverse les **trois étages réguliers/hors-contexte** ; la chaîne morpho va de la surface à l'arbre puis au BUTA. Le pipeline n'instancie que les apps — aucune logique d'automate n'y est réécrite (gate respectée).

5.2 Q5.1/Q5.2/E5.3 — Classes d'équivalence de L_1

Théorème (Myhill–Nerode). $u \approx_L v$ ssi $\forall w, uw \in L \Leftrightarrow vw \in L$. L est régulier ssi $\approx_L$ est d'indice fini, et cet indice est le nombre d'états de l'AFD minimal de L .

Mesure réelle avec témoins $S = \{\varepsilon, r, or, a\}$ sur 9 mots (nerode_classes(contains_or, ...)) — exactement **3 classes** :

signature (ε, r, or, a)	mots	classe	état AFD
(0, 0, 1, 0)	"", a, oa	« rien d'armé »	A
(0, 1, 1, 0)	o, ao, oo	« o armé »	B
(1, 1, 1, 1)	or, aor, ror	« or vu » (absorbant)	C

Le lien chiffré : **3 classes = 3 états** de l'AFD minimal du Jour 1. Le critère Q5.2 : deux états sont fusionnables à la minimisation ssi ils sont indistinguables par tous les suffixes — c'est exactement l'égalité des signatures ; la minimisation de Moore (E1.2) **calcule** les classes de Nerode en partant de la partition finaux/non-finaux et en raffinant par les témoins d'une lettre. Le fil rouge se referme : `equivalent("o", "ao") = True`, `equivalent("o", "a") = False` (test `test_nerode_trois_classes`).

5.3 Q3.5 + Bonus — Hash-consing : mesures réelles

corpus	mots	total	uniques	compression
test (2 mots, re-structuration)	2	16	9	0,438

corpus	mots	total	uniques	compression
corpus_A (préfixant, 8 préfixes appris)	5 004	33 916	6 133	0,819
corpus_B (suffixant, 9 suffixes appris)	5 004	33 916	10 581	0,688
A ∪ B (~10 000 mots)	9 452	65 052	15 045	0,769

(compression = $1 - \text{uniques}/\text{total}$; chaque mot est segmenté par `discover + segment_to_tree` puis interné dans un unique `CompactStore`.)

Discussion. Le partage est massif dès que le vocabulaire est morphologiquement riche : les chaînes d’affixes (`prefixes(mu, nil)`, `suffixes(lar, nil)`, ...) et les racines répétées ne sont stockées qu’une fois. Nos chiffres montrent ~82 % de compression sur le corpus préfixant contre ~69 % sur le suffixant (les suffixes appris y sont plus nombreux et se combinent en chaînes plus variées, donc moins partageables). Pour une langue **isolante** (type anglais, mots ≈ racines nues), presque chaque arbre serait `word(nil, rest(root, nil))` à racine distincte : partage faible, limité aux nœuds `nil`. Pour une langue **agglutinante** (turc, finnois), les longues chaînes de suffixes régulières se répètent d’un mot à l’autre : c’est là que le DAG gagne le plus — conformément à la motivation des dictionnaires morphologiques minimaux (Daciuk et al. 2000).

6. Difficultés rencontrées & choix de conception

- **E2.2 (génération CFG)** : la double borne (terminaux et non-terminaux) était indispensable — la version naïve divergeait sur $S \rightarrow SS$, exactement le piège annoncé. Nous avons aussi mémorisé les formes déjà vues (`seen`).
- **Minimisation (E1.2)** : compléter l’automate avec un état puits *avant* le raffinement (via `_completed()` fourni) ; sans cela les signatures de blocs sont fausses dès qu’une transition manque.
- **SUB (E4.3)** : nous avons choisi un appariement par les **extrémités** (effacer à droite, puis à gauche) plutôt que le marquage X de l’énoncé : la table est plus courte (11 transitions) et la sortie reste contiguë ; la fin de `m` se détecte bien par un blanc, pas par un compteur.
- **DFA.run** renvoie `None` sur transition manquante (mot rejeté) — pas d’exception : c’est ce que suppose `accepts` (`None` $\notin$ `accept`).
- **encode** : tri systématique (transitions, ensembles) + `sort_keys=True` pour un encodage canonique, sinon le round-trip n’est pas déterministe.
- **Clé canonique du hash-consing** : (`symbol, label, kids_ids`) — oublier `label` fusionnerait `root("livre")` et `root("jou")` et casserait le round-trip.
- **MUL monolithique** : le point dur était la zone résultat — sans séparateur = écrit **dès l’initialisation**, le nettoyage final ne sait pas où s’arrêter quand `m = 0` (la machine partirait à droite sur des blancs sans jamais s’arrêter). D’où les deux états d’init `qi/qr`.
- Environnement Windows : lancer avec `py` (le python du Store est un stub) ; `py -m pip install pytest` puis `py -m pytest -q` depuis `formlang_projet/`.

7. Répartition du travail

John Ulrich a piloté les jours 1 et 2 (**régulier** et **hors-contexte**) ; Gregoire a pris en charge le jour 3, le plus gros morceau isolé du barème (5 pts +2 bonus, le pivot arbres). Les jours 4 et 5 (calculabilité et intégration) — les plus denses en débogage croisé (machine universelle, pipeline bout-en-bout) — ont été menés **ensemble**. Répartition en points sur les jours 1-5 + bonus : $\approx 10/26$ (38 %) pour John Ulrich, $\approx 7/26$ (27 %) pour Gregoire, $\approx 9/26$ (35 %) en binôme.

Volet	Points	Pilote	Rôle du binôme
Jour 1 — régulier & transducteurs (dfa, nfa, fst, detector) + Q1.1–Q1.3 + bonus Thompson	4 (+1)	AGBESSI John Ulrich	relecture, tests
Jour 2 — hors- contexte (pda, cfg) + Q2.1– Q2.3 + bonus CYK	4 (+1)	AGBESSI John Ulrich	relecture, tests
Jour 3 — arbres, pivot (tree, morpho, shield, hashcons) + Q3.1–Q3.5 + bonus infixation & minimisation BUTA	5 (+2)	LATTOUNDJI Gregoire	relecture, tests
Jour 4 — calculabilité (turing, utm, mtu) + Q4.1– Q4.4 + bonus MUL monolithique	4 (+1)	Binôme (ensemble)	—
Jour 5 — intégration & Myhill–Nerode (pipeline, myhill_nerode) + Q5.1–Q5.2 + bonus mesure 10 000 mots	3 (+1)	Binôme (ensemble)	—
Débogage croisé, vérification	—	Binôme (ensemble)	—

Volet	Points	Pilote	Rôle du binôme
finale (37 tests verts) & mise en place du dépôt Git			
Rédaction du rapport, mesures, mise en forme	—	AGBESSI John Ulrich (rédaction principale)	LATOOUNDJI Gregoire (relecture, vérification des chiffres)
Soutenance (10 min)	—	J. U. AGBESSI présente J1, J2 et la démo bout- en-bout	G. LATOOUNDJI présente J3 (le pivot arbres) ; J4 et J5 présentés à deux

À ajuster avant le rendu si la répartition réelle a différé — l’important pour le jury est que chacun puisse défendre **l’intégralité** du code en soutenance, pas seulement « sa » part.

Annexe — sortie console

```
$ py -m pytest -q
.....
[100%]
37 passed in 0.35s
```

(28 tests de base + 9 tests bonus de tests/test_bonus.py.)

```
$ py pipeline.py
== démo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isolé)
BLOQUÉ  sys(role)
BLOQUÉ  seq(frame(ovr),txt)
BLOQUÉ  sys(seq(txt,frame(role)))
```

```
$ py pipeline.py --word 4or
      normalisé(FST) : aor
      facteur_or(AFD) : True
      délimiteurs_ok(PDA) : True
```

```
$ py pipeline.py --morpho mufafak
{'classe(BUTA)': 'PREFIXED'}
```